

1 Лекция 1

1.1 Выпуклость

Ниже потребуется определение выпуклости функции и множества.

Определение 1. Подмножество $C \subset \mathbb{R}^n$ вещественного векторного пространства называется выпуклым если

$$\lambda x + (1 - \lambda)y \in C \quad \forall x, y \in C, \lambda \in [0, 1].$$

Функция $f : C \rightarrow \mathbb{R}$, определённая на выпуклом множестве, называется выпуклой если она удовлетворяет неравенству Йенсена

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y).$$

В оптимизации свойство выпуклости важно, поскольку оно гарантирует глобальность локальных минимумов.

Определение 2. Пусть $f : \mathbb{R}^n \supset D \rightarrow \mathbb{R}$ — функция, определённая на подмножестве вещественного векторного пространства. Точка $\hat{x} \in D$ называется локальным минимумом если существует окрестность U точки $\hat{x}$ такая, что

$$f(x) \geq f(\hat{x}) \quad \forall x \in U \cap D.$$

Минимум $\hat{x}$ называется глобальным если

$$f(x) \geq f(\hat{x}) \quad \forall x \in D.$$

Лемма 1. Пусть $f : D \rightarrow \mathbb{R}$ — выпуклая функция, определённая на выпуклом множестве. Тогда любой локальный минимум $\hat{x} \in D$ функции f является глобальным.

Доказательство. Пусть $x \in D$ — произвольная точка. Тогда для любого $\lambda \in [0, 1]$ имеем

$$f(\lambda x + (1 - \lambda)\hat{x}) \leq \lambda f(x) + (1 - \lambda)f(\hat{x}).$$

Для всех $\lambda \in [0, 1]$ имеем $\lambda x + (1 - \lambda)\hat{x} \in D$ и для достаточно малых $\lambda > 0$ эта точка лежит в любой заданной наперёд окрестности точки $\hat{x}$. Так как $\hat{x}$ — локальный минимум, для достаточно малых λ имеем

$$f(\lambda x + (1 - \lambda)\hat{x}) \geq f(\hat{x}).$$

Комбинируя, получим

$$f(\hat{x}) \leq f(x),$$

что и требовалось доказать. □

1.2 Классификация задач

Формализованная задача оптимизации имеет вид

$$\min_x f_0(x) : \quad f_i(x) = 0, \quad g_j(x) \leq 0, \quad i \in I, \quad j \in J \quad (1)$$

где

- $x \in \mathbb{R}^d$ — переменная
- f_0 — функционал цены
- f_i — функции-ограничения типа равенства
- g_j — функции-ограничения типа неравенства

Множество

$$X = \{x \mid f_i(x) = 0, g_j(x) \leq 0, i \in I, j \in J\}$$

называется *допустимым множеством* задачи.

Если $X = \emptyset$, задача называется *недопустимой*, и её оптимальное значение равно $+\infty$, в ином случае *допустимой*.

Задача называется *строго допустимой*, если существует точка x , для которой $f_i(x) = 0$ для всех $i \in I$ и $g_j(x) < 0$ для всех $j \in J$.

Задача называется *безусловной*, если $I = J = \emptyset$, иначе *обусловленной*.

Задача называется *гладкой* порядка k если функции f_0, f_i, g_j принадлежат классу C^k k раз непрерывно дифференцируемых функций. В безусловной оптимизации встречаются также классы μ -сильно выпуклых функций, выполняющих условие

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y) - \frac{\mu}{2}\lambda(1 - \lambda) \cdot \|x - y\|^2 \quad \forall x, y; \lambda \in [0, 1],$$

а также класс функций с L -липшицевым градиентом, выполняющим условие

$$\|\nabla f(x) - \nabla f(y)\| \leq L \cdot \|x - y\| \quad \forall x, y.$$

Если μ -сильно выпуклая функция f непрерывно дифференцируема, то она для каждого $\hat{x}$ имеет миноранту

$$f(x) \geq f(\hat{x}) + \langle \nabla f(\hat{x}), x - \hat{x} \rangle + \frac{\mu}{2}\|x - \hat{x}\|^2.$$

Допустимая задача называется *неограниченной* если её оптимальное значение $\inf_{x \in X} f_0(x)$ равно $-\infty$, в ином случае *ограниченной*.

Если задача допустимая и ограниченная, то всё равно точки $x^* \in X$, на которой инфимум достигается, т.е., оптимального решения, может не существовать.

Пример: В задаче

$$\min_{x \in \mathbb{R}^2} x_2 : \quad x_1 \geq 0, x_1 x_2 \geq 1$$

оптимальное значение равно 0, но оптимального решения нет.

Функционал цены добавлением дополнительной переменной можно сделать линейным. Задача

$$\min_x f_0(x) : \quad f_i(x) = 0, g_j(x) \geq 0$$

эквивалентна задаче

$$\min_{\tau, x} \tau : \quad f_i(x) = 0, g_j(x) \leq 0, \tau \geq f_0(x)$$

с линейной ценой.

Класс задач (1) очень широкий. Например, условие $x_i^2 = x_i$ отражает бинарность переменной x_i , а условие $\sin(x_i \pi) = 0$ — её целочисленность. Поэтому построить универсальный алгоритм, решающий все задачи этого класса, не представляется реалистичным.

Иногда ограничения в задаче оптимизации не записываются напрямую в форме (1), например, если речь идёт о матричном неравенстве. Но даже в этом случае существуют эквивалентные формы вида (1).

Матричное неравенство имеет вид

$$A(x) \succeq 0,$$

где $A : \mathbb{R}^n \rightarrow \mathcal{S}^m$ есть оператор из пространства переменных в пространство симметричных матриц. Условие состоит в том, что образ допустимой точки x должен являться неотрицательно определённой матрицей, т.е., $\lambda(A(x)) \geq 0$ (собственные значения неотрицательны). Если оператор A аффинный, то мы имеем дело с *линейным матричным неравенством*.

В этом курсе в основном будут рассматриваться итеративные методы, которые строят последовательность точек на основе локальной информации, т.е., о свойствах функций f_0, f_i, g_j в окрестности ранее сгенерированных точек x^k . Такие алгоритмы страдают недостатком, что не могут отличить глобального минимума от локального.

Определение 3. Глобальным минимумом для задачи (1) называется точка $x^* \in X$ такая, что $f_0(x^*) \leq f(x)$ для всех $x \in X$. Здесь X — допустимое множество задачи.

Локальным минимумом для задачи (1) называется точка $\hat{x} \in X$ такая, что существует окрестность U точки $\hat{x}$ в $\mathbb{R}^d$ такая, что $f_0(x^*) \leq f(x)$ для всех $x \in X \cap U$.

Упомянутая проблема разрешается в случае, когда задача выпукла.

Определение 4. Задача (1) называется выпуклой если $f_0, g_j, j \in J$ — выпуклые функции, а $f_i, i \in I$ — аффинные функции.

В этом случае допустимое множество X задачи также выпукло, и любой локальный минимум является глобальным в силу леммы 1. Следует отметить, что выпуклость задачи является достаточным, но не необходимым условием того, что любой локальный минимум является глобальным.

Например, в задаче

$$\min_{x \in \mathbb{R}^2} x_2 : x_1 \geq 0, x_2 \geq x_1^3$$

допустимое множество и функционал выпуклы, но ограничение $-x_2 + x_1^3 \leq 0$ — невыпукло.

Однако, для построения эффективных алгоритмов нужна именно выпуклость функционала и каждого ограничения по отдельности. Установить выпуклость или даже пустоту допустимого множества в общем случае — НП-сложная задача.

Существуют методы решения, использующие нелокальную информацию о задаче. Такие методы применяются для задач, обладающих богатой структурой, например, *линейные программы*, в которых все функции f_0, f_i, g_j аффинны.

Функционал цены и ограничения могут быть заданы разными способами. Например, они могут задаваться явным выражением. Но часто они задаются некоторым алгоритмом, детали которого могут быть неизвестны, который на вход получает аргумент x , а на выход подаёт значения $f_0(x), f_i(x), g_j(x)$ и/или производные этих функций. Такой алгоритм также называют *оракулом*. Если оракул выдаёт производные до порядка k включительно, говорят, что он порядка k . Таким образом, оракул нулевого порядка выдаёт только значение функций. Бывают и оракулы другого типа, например, принимающие на вход два аргумента x, x' и выдающие информацию о том, какое из неравенств $f(x) \leq f(x')$ или $f(x') \leq f(x)$ справедливо.

Выделим некоторые классы задач:

- если все функции f_0, f_i, g_j в описании задачи аффинны, то задача является *линейной программой* (ЛП)
- если функционал цены f_0 квадратичен, а функции-ограничения f_i, g_j — аффинны, то задача является *квадратичной программой* (КП)
- если функционал f_0 аффинный, но кроме аффинных функций-ограничений есть ещё условия на целочисленность части переменных, то задача является (*смешанно-целочисленной линейной программой*) (ЦЛП)
- если функционал f_0 аффинный, но кроме линейных равенств и неравенств присутствуют ещё линейные матричные неравенства, то задача является *полуопределённой программой*

Линейные и полуопределённые программы являются выпуклыми и относятся к классу конических программ. Они хорошо изучены и существует специализированное программное обеспечение, решающее такие задачи довольно эффективно. Полуопределённые программы возникают в управлении и идентификации параметров динамических систем. Квадратичные программы выпуклы, если выпукла квадратичная функция цены. Выпуклые задачи КП решаются теми же методами, что и ЛП, с некоторыми модификациями. Задачи ЦЛП сложные, и часто возникают в промышленных приложениях. Для их решения также есть специализированное программное обеспечение (решатели).

1.3 Сложность алгоритма и класса задач

Материалы из этого раздела можно подробнее изучить по монографии Ю.Е. Нестерова "Методы выпуклой оптимизации" <https://mipt.ru/dcam/upload/abb/nesterovfinal-arpgzk47dcy.pdf>

Сложность алгоритма решения задачи обычно измеряется числом арифметических операций, которых алгоритм выполняет до выдачи решения. Однако, определять сложность задачи как сложность наиболее простого алгоритма, который её решает, не имеет смысла. Дело в том, что для каждой конкретной оптимизационной задачи найдётся очень простой алгоритм, который её решает, а именно, выдающий решение задачи в качестве единственной операции, которую он проделывает. Поэтому осмысленным может быть только сложность алгоритма, решающего целый класс задач.

Точное решение x^* и оптимальное значение $f_0(x^*)$ задачи редко удаётся получить, и они в подавляющем большинстве случаев и не нужны. В общем случае алгоритм итеративный и стремится к решению асимптотически. Определение того, с какой точностью задача решена, может быть проведено разными способами. В зависимости от этого говорят о разных видах сходимости алгоритма:

- сходимость по функционалу $f_0(x) - f_0(x^*) \leq \epsilon$
- сходимость по аргументу $\|x - x^*\| \leq \epsilon$
- сходимость по норме градиента $\|\nabla f_0(x)\| \leq \epsilon$

Последнее определение сходимости имеет смысл только для безусловных выпуклых задач, так как в иных случаях градиент в оптимальной точке может быть ненулевым.

Множество задач, которым свойственны какие-то общие характеристики, называют *моделью*. Конкретная задача в модели определяется данными, например, функциями, определяющими функционал и ограничения. Модель может также содержать параметры, например, размерность пространства переменных или точность ϵ , с которой должна решиться задача. От алгоритма разумно потребовать, чтобы он был способен решать все задачи данной модели, а его сложность определить как максимальное количество операций, которые он должен выполнить для решения задачи с данным набором параметров. Задача, для решения которой потребуется это максимальное количество, как правило, зависит от самого алгоритма. Поэтому нет смысла говорить о самой сложной задаче класса.

Для более детального анализа рассмотрим работу алгоритма подробнее. Для решения поступившей задачи алгоритм располагает априорной информацией о модели. Мы рассматриваем *локальные* и *итеративные* алгоритмы, которые, начиная с некоторой точки x^0 , на каждом шаге генерируют новую точку x^{k+1} на основе информации, полученной о функциях, определяющих задачу, в окрестности ранее сгенерированных точек. Обычно это производные функции в этих точках до некоторой степени.

Чтобы отделить компоненту сложности алгоритма, зависящую от устройства самого алгоритма, от сложности, связанной с представлением функций, задающих задачу, полезно ввести понятие *оракула*. Оракул — это чёрный ящик, который на вход получает очередную точку x^k , а на выходе презентует определённую информацию об инстанции задачи в окрестности точки x^k . Можно анализировать и сравнивать алгоритмы, использующие один и тот же оракул, но генерирующие последовательность точек по-разному.

Так как ответы оракула являются для алгоритма единственным источником информации о конкретной инстанции задачи, понятие о наихудшей задаче для алгоритма можно перенести на оракул, который в этом случае называется *сопротивляющимся*. Взаимодействие алгоритма и сопротивляющегося оракула можно представить в виде игры двух игроков, которые чередуют свои ходы. Алгоритм генерирует новую точку, в то время как оракул генерирует локальную информацию в этой точке. Сопротивляющийся оракул при этом формирует свой ход с целью максимально затормозить сходимость решений, выдаваемых алгоритмом. Последовательность ответов этого оракула, таким образом, зависит от выбора точек x^k алгоритмом. Проблема поиска *наилучшего* алгоритма поэтому по сути седловая (минимаксная) задача: алгоритм должен максимизировать скорость сходимости, играя против сопротивляющегося оракула. Заметим, что последний должен генерировать свои ответы с учётом условия, что существует инстанция задачи, с которой эти ответы совместимы.

Иногда удаётся получить нижнюю границу сложности класса задач, построив сопротивляющийся оракул, который препятствует работе любого алгоритма на этом классе. Если алгоритм имеет сложность на своём сопротивляющемся оракуле, равную нижней границе, то это доказывает оптимальность алгоритма.

Пример: Рассмотрим задачу минимизации липшицевой функции, т.е., функции, удовлетворяющей условию

$$|f(x) - f(y)| \leq M\|x - y\|_2 \quad \forall x, y \in X, \quad (2)$$

на множестве $X \subset \mathbb{R}^d$ объёма V с помощью оракула 0-го порядка, т.е., выдающего в ответ на вход $x \in X$ только значение функции $f(x)$. При этом критерием точности решения будет разница в значениях функционала на построенном решении и в оптимальной точке.

Пусть оракулу были презентованы точки $x_0, \dots, x_N \in X$, в ответ на которые он выдал значения

$$v_k = f(x_k). \quad (3)$$

Найдём наихудшую функцию, совместную с ответами оракула. Эта функция должна удовлетворять условиям липшица (2) и ответам оракула (3), при этом максимизируя ошибку

$$\min_k v_k - \inf_{x \in X} f(x).$$

Очевидно, функция

$$f(x) = \max_k (v_k - M\|x - x_k\|_2)$$

решает эту задачу. Ошибка по значению функционала задаётся

$$\min_k v_k - \inf_{x \in X} \max_k (v_k - M\|x - x_k\|_2) = \min_k v_k + \sup_{x \in X} \min_k (M\|x - x_k\|_2 - v_k) \leq M \sup_{x \in X} \min_k \|x - x_k\|_2.$$

Неравенство превращается в равенство если все v_k одинаковы. В этом случае точность решения задачи ограничена линейной функцией от максимального расстояния от точки множества X до множества $\{x_k \mid k = 0, \dots, N\}$, т.е., радиусом наибольшего шара с центром в X , который не содержит ни одной точки x_k .

Следовательно, сопротивляющийся оракул в качестве значения функции должен выдавать постоянное значение, без ограничения общности 0. Пусть на итерации n были построены точки $x^0, \dots, x^{n-1} \in X$. Пусть ρ — радиус шара с объёмом $\frac{V}{n+1}$ в $\mathbb{R}^d$. Тогда существует точка $\hat{x} \in X$, для которой расстояние до любой из точек $x^0, \dots, x^{n-1}$ не меньше, чем ρ . Действительно, построив вокруг каждой из точек x^k шар радиуса ρ , мы этими шарами покроем в $\mathbb{R}^d$ объём не больше, чем $\frac{n}{n+1}V < V$. В частности, останутся точки из X , не лежащие ни в одном из шаров.

Положим $\epsilon = \rho M$ и построим функцию

$$f(x) = \begin{cases} -\epsilon \cdot M \cdot \|x - \hat{x}\|, & \|x - \hat{x}\| \leq \rho, \\ 0, & \|x - \hat{x}\| \geq \rho. \end{cases}$$

Эта функция удовлетворяет условию (2), её минимальное значение равно $-\epsilon$, а значение функционала в выданной алгоритмом точке равно 0. Поэтому любой алгоритм решает задачу с ошибкой, не меньшей

$$\epsilon = \left(\frac{\Gamma(\frac{d}{2} + 1)V}{\pi^{d/2}(n+1)} \right)^{1/d} = O(n^{-1/d}).$$

Иными словами, для гарантированного достижения точности ϵ любому алгоритму нужно выполнить не менее $n = O(\epsilon^{-d})$ итераций.

С другой стороны, если X имеет не слишком сложную геометрию, например, является кубом, то покрыв множество X равномерной сеткой точек и вызвав оракул на всех этих точках, можно ограничить ошибку ϵ тем же выражением $O(n^{-1/d})$. Из этого следует, что данный алгоритм оптимален для рассматриваемого класса задач с точностью до постоянной.

1.4 Одномерный поиск

Более подробно с материалом этого пункта можно ознакомиться в пп. 2.2.1 пособия [1] (<https://arxiv.org/pdf/2106.01946.pdf>).

Одномерный поиск с помощью бисекции: Рассмотрим проблему минимизации унимодальной функции $f : \mathbb{R} \supset I \rightarrow \mathbb{R}$ на интервале I , конечном или бесконечном. Здесь унимодальность означает, что f строго

убывает для $x \leq x^*$ и строго монотонно возрастает для $x \geq x^*$, где x^* является минимум функции. Предположим, что минимум x^* существует и что есть оракул, который для любой точки x выдаёт какой из случаев $x \leq x^*$ или $x \geq x^*$ имеет место, например, вычислив знак производной $f'(x)$ в случае дифференцируемой функции. Точность решения определим как расстояние от достигнутой точки до решения задачи, т.е., по аргументу.

Рассмотрим сначала случай конечного интервала $I = [a, b]$. Решение x^* в начале может находиться в любой точке этого интервала. Вызов оракула в точке $x^0 \in (a, b)$ позволяет определить, находится ли x^* в интервале $[a, x^0]$ или в интервале $[x^0, b]$. На каждом шаге можно таким образом поделить текущий интервал на два подинтервала и выбрать один из них. Если длина интервала станет не больше, чем ϵ , то решение определено с этой точностью.

Построим сопротивляющийся оракул. Каждый раз, когда текущий интервал делится на две части вызовом оракула в очередной точке x^k , ответ оракула будет таким, что больший из двух подинтервалов становится текущим. Таким образом, длина интервала спустя n шагов не будет меньше, чем $\frac{b-a}{2^n}$. Для достижения точности ϵ любому алгоритму придётся сделать не менее, чем $n = \lceil \log_2 \frac{b-a}{\epsilon} \rceil$ шагов. Это выражение есть нижняя граница на сложность класса задач.

С другой стороны, выбор центра интервала, или метод бисекции (дихотомии), гарантирует, что потребуется не более, чем $\lceil \log_2 \frac{b-a}{\epsilon} \rceil$ шагов для достижения точности ϵ . Таким образом, метод бисекции является оптимальным для данной задачи. Вместе с описанным выше сопротивляющимся оракулом метод бисекции составляет минимаксное решение.

Рассмотрим теперь модификацию, в которой начальный интервал является лучом с крайней точкой a , открытым вправо. В этом случае легко построить сопротивляющийся оракул, который гарантирует, что интервал, в котором находится решение x^* , на любом шаге будет неограниченным. Для этого оракул на любой вход x^k должен выдавать ответ $x^* > x^k$. Ясно, что в предположении существования x^* нет функции f , генерирующей соответствующую последовательность ответов оракула. Однако, для любого конечного n есть функция, для которой ответы сопротивляющегося оракула совпадают с первыми n из реальных ответов. Поэтому решение задачи с заданной точностью не может быть гарантировано за n шагов, сколь бы большим n ни было.

Для того, чтобы ограничить сложность сверху, нужно ввести дополнительный параметр $R = |x^0 - x^*|$. Если R известен, то задача по сути сводится к предыдущей с компактным интервалом, и сложность ограничена сверху числом $n = \lceil \log_2 \frac{R}{\epsilon} \rceil$ вызовов оракула. Если R неизвестно, то можно генерировать точки по формуле $x^k = a + \epsilon \cdot 2^k$, пока оракул не выдаст ответ $x^* < x^k$. Ясно, что этот момент наступит спустя не более, чем $\lceil \log_2 \frac{R}{\epsilon} \rceil$ итераций, и общее число итераций не более, чем удваивается.

Одномерный поиск с помощью метода золотого сечения: Предположим, что мы снова желаем минимизировать унимодальную функцию на интервале $[a, b] \subset \mathbb{R}$, но на этот раз оракул в ответ на вход x^k выдаёт значение функции $f(x^k)$, т.е., является оракулом 0-го порядка.

Пусть алгоритм минимизации выбрал точки $x^0 = a$, $x^1 = b$, $x^2 \in (a, b)$ и вызвал оракул на этих точках. Если $f(x^2) < f(x^0)$ и $f(x^2) < f(x^1)$, то точка x^* всё ещё может лежать во всём открытом интервале (a, b) . Для того, чтобы сузить интервал, нужно знать значение функции в двух различных внутренних точках. Тогда можно отбросить один из крайних подинтервалов, оставив два других. Во внутренности большего из них генерируется новая точка, в которой вызывается оракул. Для того, чтобы соотношение длин интервалов оставалось постоянным, необходимо делить интервал золотым сечением, т.е., в пропорции $\alpha : 1 - \alpha$, где $\alpha = \frac{\sqrt{5}-1}{2}$. На каждом шаге длина интервала уменьшается на фактор α . Таким образом, для достижения точности ϵ по аргументу алгоритм должен сделать $\approx \log_{\alpha^{-1}} \frac{|b-a|}{\epsilon}$ шагов.

Список литературы

- [1] Е.В. Воронцова, А.В. Гасников, Р.Ф. Хильдебранд, and Ф.С. Стонякин. *Выпуклая оптимизация*. МФТИ, 2021.